

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Americo Ignacio Acuña Rodríguez

8 de septiembre de 2026

17:44

1. Introducción

El análisis teórico habitual clasifica los algoritmos por cómo crecen sus operaciones en el papel, pero deja de lado factores reales del computador como la memoria caché o las optimizaciones del compilador. En este trabajo se ponen a prueba dos problemas mediante experimentos prácticos: ordenar arreglos (MergeSort, QuickSort, PatienceSort y `std::sort`) y multiplicar matrices cuadradas (Naive contra Strassen), comparando la teoría con su rendimiento real. El código fuente y los datos utilizados se encuentran en: https://github.com/Ameripro2006/INF221_T1_2026-2.

2. Entorno experimental

Los algoritmos (desarrollados en C++17 y compilados con `g++ -O2`) se ejecutaron en un entorno Linux (Kernel 6.x) con un procesador AMD Ryzen 5 7600X y 32 GB de RAM DDR5. Los tiempos de ejecución se midieron utilizando `std::chrono::high_resolution_clock`, y el consumo de memoria máxima (Peak RSS) se registro mediante la llamada al sistema `getrusage`.

3. Resultados y Análisis

3.1. Algoritmos de Ordenamiento

El análisis experimental muestra una fuerte sensibilidad a la distribución inicial de los datos.

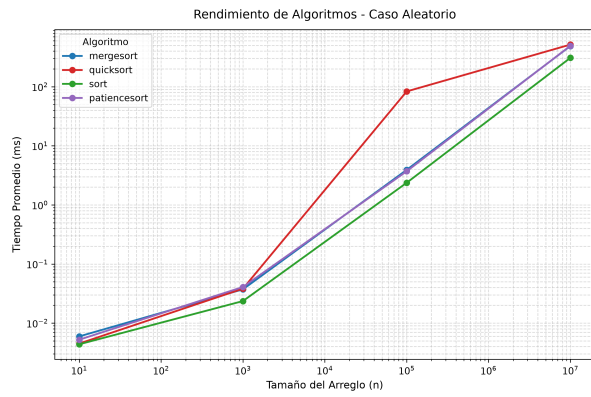


Figura 1: Tiempo: Caso aleatorio.

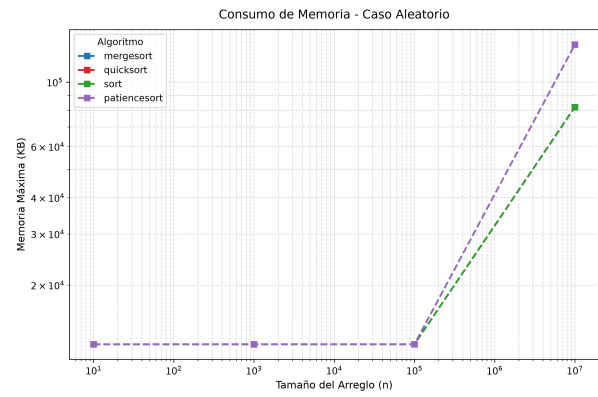


Figura 2: Memoria: Caso aleatorio.

En la Figura 1, `std::sort` es el más rápido. QuickSort se vuelve muy lento con arreglos ordenados porque su tiempo sube a $\mathcal{O}(N^2)$, lo que agota la memoria de la pila (*Stack Overflow*) o supera el tiempo límite para $N \geq 10^6$. MergeSort mantiene su comportamiento teórico de $\mathcal{O}(N \log N)$, mientras que PatienceSort también lo cumple, pero demora más tiempo por tener operaciones internas más pesadas.

Respecto a la memoria (Figura 2), `std::sort` es el más eficiente al trabajar directamente sobre el mismo arreglo (*in-place*), mientras que MergeSort y PatienceSort gastan más memoria a medida que crece el tamaño ($\mathcal{O}(N)$) debido a las estructuras auxiliares que usan.

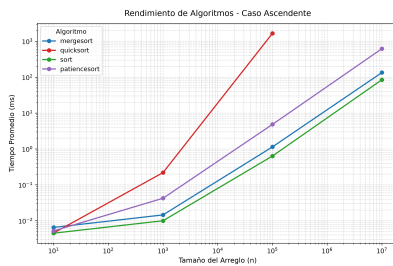


Figura 3: Ascendente.

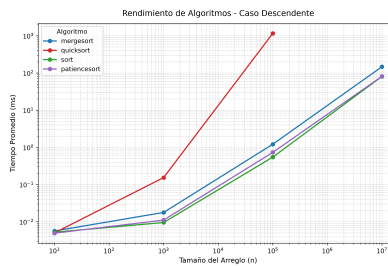
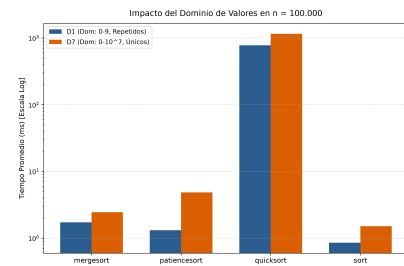


Figura 4: Descendente.

Figura 5: Dominio (10^5).

3.2. Multiplicación de Matrices

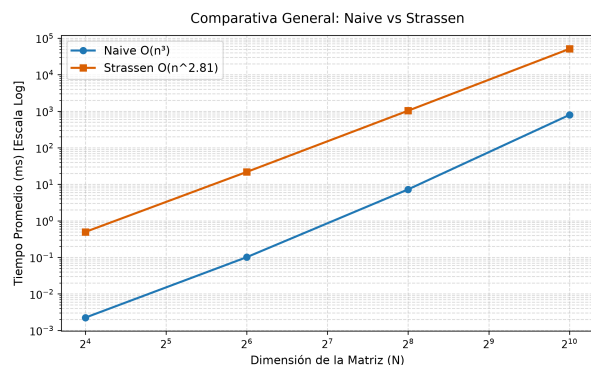


Figura 6: Tiempo: Naive vs Strassen.

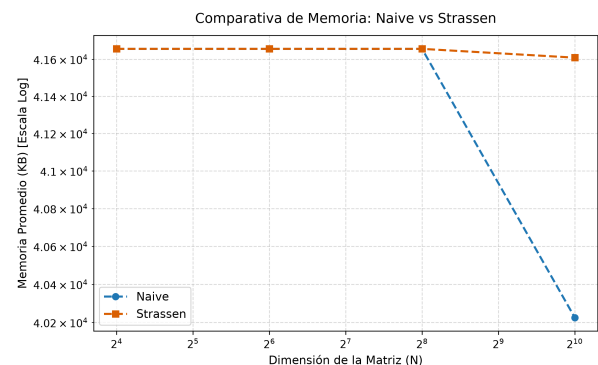


Figura 7: Memoria: Naive vs Strassen.

Naive es muchísimo más rápido que Strassen (Figura 6). Al llegar a $N = 1024$, tantas llamadas recursivas (cerca de $3,29 \times 10^8$) generan un costo extra enorme que termina anulando cualquier ventaja teórica del algoritmo. Esto también se nota en la memoria (Figura 7): Strassen pide memoria dinámica constantemente (*heap*), mientras que Naive mantiene un uso fijo y ordenado de $\mathcal{O}(N^2)$. Además, el tiempo de ejecución no cambió según el tipo o los valores de las matrices, ya que ninguno de los dos algoritmos se salta operaciones cuando hay ceros.

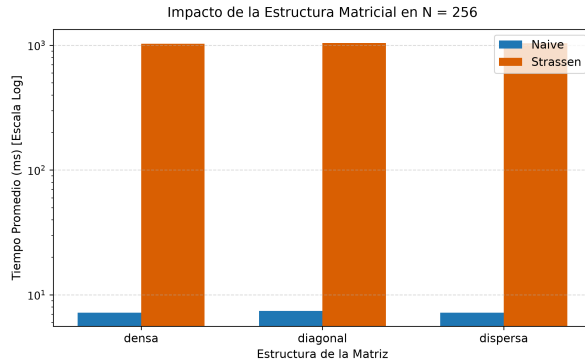


Figura 8: Estructura interna.

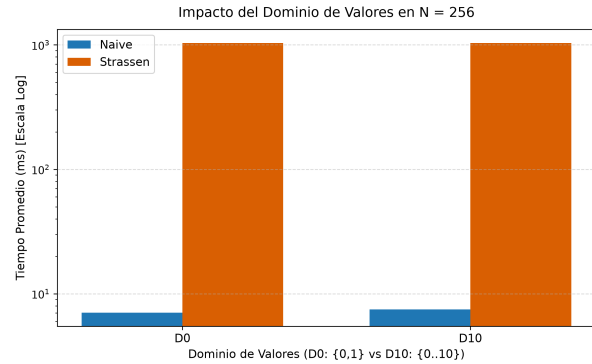


Figura 9: Dominio numérico.

4. Conclusiones

La notación asintótica teórica es insuficiente en entornos reales sin considerar la arquitectura. En el ordenamiento, el orden inicial condiciona fatalmente la partición ingenua en QuickSort (causando *Stack Overflow*) y el número de pilas en PatienceSort, destacando la estabilidad de MergeSort y `std::sort` (*in-place*). En matrices, la ventaja teórica de Strassen ($\mathcal{O}(N^{2,81})$) es anulada en $N \leq 1024$ por el gran costo de gestión dinámica de memoria y la degradación de localidad frente al consecutivo de Naive, fuertemente optimizado por el compilador.